

Eloquent: Improving Text Editing for Mobile

Olivier Bau
32AL
olivier@32al.io

Scott Jenson
Google
scott@jenson.org

ABSTRACT

We present Eloquent, an exploration of new interaction techniques for text editing on mobile. Our prototype combines techniques for targeting, selection, and command menus. We demonstrate preliminary results in the form of a text editor, whose design was informed by feedback from users of existing systems. With Eloquent, selecting and acting on text can be done with a single gesture for both novice and expert users.

CCS CONCEPTS

• **Human-centered computing** → Pointing; Text input.

KEYWORDS

Mobile, text editing, touch displays, menus

ACM Reference Format:

Olivier Bau and Scott Jenson. 2021. Eloquent: Improving Text Editing for Mobile. In *The Adjunct Publication of the 34th Annual ACM Symposium on User Interface Software and Technology (UIST '21 Adjunct)*, October 10–14, 2021, Virtual Event, USA. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3474349.3480178>

1 INTRODUCTION

Creating text on mobile is composed of two separate activities: text input and text editing. Text input has significantly improved over the last 10 years thanks to efficient keyboards and auto-correction[1]. Unfortunately, text editing on mobile phones can be problematic [2, 7]. This project, Eloquent, focuses on improving text editing, which consists of three distinct steps: placing the cursor, selecting text, and acting on that text.

We asked 10 Android users to do a simple exploratory study, running each one through a series of text editing tasks to place the cursor accurately, delete a word, or copy and paste a sentence. In addition we asked general questions about their thoughts and approach to text editing on their phones.

One of the tasks was to remove a double space between words. This was problematic for every user, with one taking 19 taps to complete the task. A more typical subject needed 5 attempts. This is partially due to occlusion errors [2], but there was also an accidental menu popup and an accidental handle drag. Using a single tap event for multiple functions can misunderstand a user's intent easily and is likely a source of confusion.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

UIST '21 Adjunct, October 10–14, 2021, Virtual Event, USA

© 2021 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-8655-5/21/10.

<https://doi.org/10.1145/3474349.3480178>

Every subject expressed deep frustration with the entire process of composing and editing text on a phone, limiting themselves to simple shorter text input bursts and avoiding substantial text creation. Many preferred to delete and retype rather than to attempt editing. Eloquent, aims to improve text editing by improving character targeting, text selection, and menu use.

2 RELATED WORK

There is significant work studying mobile text input, for example Shumin et al.'s keyboard significantly improves text input speed [1], while Leiva et al. focused on how to make smaller keyboards more accurate [3]. Related work focuses primarily on subtasks of text editing separately, i.e. targeting, selection, or command evocation. Zheng et al. investigated marking menus for mobile, enabling efficient command invocation on touch displays [8]. Clark discusses the negative consumer impact on iOS text editing issues when Apple removed, and then later added back the text magnifier [4]. Our conjecture is that mobile text editing is a combination of several interaction techniques, and few papers discuss text editing as a supportive combination. We will build upon this work to suggest unified improvements to targeting, selecting and command evocation.

3 ELOQUENT INTERACTION TECHNIQUES

Cursor placement is an essential part of text editing. It is the precursor of any editing task, from the simple deletion of a single character, to copy and pasting a sentence. While touch input presents many advantages through direct manipulation, its use becomes challenging when targeting small letters. With a draggable cursor and an improved magnifier, Eloquent hopes to improve text targeting.

3.1 Dragging the cursor

On the desktop, a mouse cursor enables users to gradually approach their target which encourages more accurate placement. Mobile requires a direct finger tap which lacks accuracy. Current systems use text handles to adjust the cursor placement after the fact. As a result, tapping can be interpreted multiple ways: place the cursor, drag the cursor, or open a popup menu. This ambiguity cascades to long press and double tap, which can fail as the first tap is interpreted incorrectly (e.g. you can't long press if the system thinks you are dragging the text handle). All of these issues make cursor placement a more challenging task.

In Eloquent, we make dragging the cursor a first-class action. We ensure that the handle is permanently visible so dragging the cursor via the handle is always possible. In addition, when dragging occurs far from the handle, the cursor handle automatically snaps to the user's finger position. As a result, the user has a uniformly consistent means to place and drag the cursor. There is no longer any ambiguity in the user's intent, all taps and drags do exactly the same thing. This behavior encourages users to drag anywhere in

- 1) Drag the text cursor, notice how the inline magnifier that is visually much simpler and less distracting than what we have today.
- 2) You don't need to tap then separately drag, you can drag horizontally anywhere to place and move the cursor at the same time. You can still scroll the text area normally.

Figure 1: Always visible and draggable text cursor

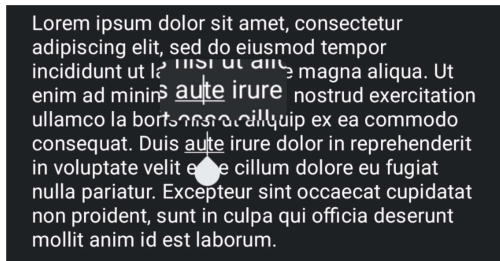


Figure 2: Existing magnifier today

- 1) Drag the text cursor, notice how the inline magnifier that is visually much simpler and less distracting than what we have today.
- 2) You don't need to tap then separately drag, you can drag horizontally anywhere to place and move the cursor at the same time. You can still scroll the text area normally.

Figure 3: Eloquent inline magnifier

the text and approach the target gradually rather than attempting a perfect tap, much like what happens on desktop today.

3.2 Magnifier

To help drag the cursor more accurately, current systems use an offset magnifier to work around finger occlusion. This works reasonably well for single lines of text but can get visually confusing in larger text fields as users have two versions of the cursor visible at once.

Because the cursor in Eloquent is dragged around by the handle, it offers the opportunity to display the magnifier inline which is visibly much calmer and allows the user to see where they are targeting, while maintaining the user's orientation within the target area. Dragging the handle in Eloquent provides enough space above the finger for the magnifier to be displayed in line. This extra space applied to dragging doesn't apply to tapping, so that behavior remains unchanged. This extra distance was immediately noticed by testers but they also quickly adjusted and reported no issues within a minute of using it.

In order to distinguish between dragging the cursor and standard vertical scrolling, Eloquent makes a quick test on the initial drag direction. If the drag vector is <30 degrees off of vertical, the gesture remains a vertical drag, otherwise it is a cursor drag event. This simple compromise appears quite effective for most usage.

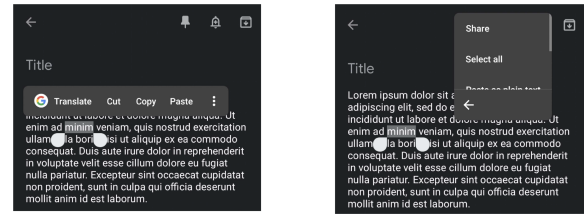


Figure 4: Today's Android Hierarchical Menu

These cursor and magnifier changes together encourage a smoother and slower targeting gesture.

4 SELECTION

Today mobile users select text by either a double tap or a long press and both have the same targeting and occlusion issues as placing the cursor. Eloquent's use of a draggable cursor initially encouraged a timeout to trigger selection but this surprised users as the draggable cursor encourages slower more accurate placement which conflicts with a timeout. We tried various types of graphical feedback to show the timer but none were effective. Instead Eloquent senses finger pressure on the display, which provides a more definitive user action.

Eloquent waits until the user pauses over a word and then looks for a harder press on the screen. This drag-pause-press sequence starts a selection.

This new event, drag-press, flows naturally from the draggable cursor into a selection action. We named it drag-press to stress the difference with previous mainstream use of pressure sensing that were focused on sensing strong vs light taps.

This event is similar to Apple's now retired 3D touch, but is only triggered while dragging. Sensing this action can be done with specific hardware, but can also be prototyped using the barometric pressure sensor present in most water-resistant phones. Pressing on the screen creates an increase in pressure that can be detected by the barometer. Eloquent keeps two different rolling averages of this pressure data, one short and the other much longer. When the faster, more reactive average exceeds the slower average by a threshold, this triggers the drag-press event. We found that this simple algorithm provided robust event sensing on devices of various sizes.

5 MENU

After placing the cursor and selecting text, a final step in text editing is often command invocation. All mobile OSs today forgo the desktop menu bar for a popup menu. To fit within the screen, this menu is often hierarchical which previous research has shown can be difficult to use[6]. In addition, the existing menu has no accelerators[7], it's the same singular approach for all users, no matter their skill level. Eloquent provides an alternative by changing how the popup menu layout and how it is invoked.

5.1 Menu Layout

The Eloquent menu is non-hierarchical, placing the most used clipboard commands on the left, and the less common commands

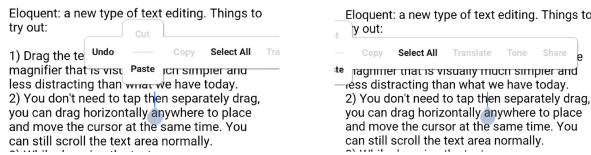


Figure 5: Eloquent's horizontal menu (open and scrolled)

scrolling off to the right. Our test subjects, without any prompting, quickly understood and were able to scroll the menu to see additional items. This menu is invoked just as today's menu, when the user lifts their finger. This allows this new menu to easily integrate with existing text editing behaviors.

Today's existing menu is often quite short and flips from flat to hierarchical with as little as 4 menu items. The Eloquent menu allows for a larger number of items before requiring a hierarchical menu. As discussed in Allen [6], reducing hierarchy in menus reduces the cognitive load in finding and targeting menu items.

5.2 Drag-based Menu Selection

Eloquent uses the drag-press gesture to lead right into using the menu. An initial drag-press starts a selection, the user can then optionally drag to extend it. An additional drag-press invokes the menu. The user's finger never leaves the screen. There is a small blue cursor displayed on the menu. While dragging the finger, the cursor moves and highlights the menu item to be invoked when the user releases their finger from the screen. Note: dragging the cursor to the right automatically scrolls the menu to the left.

5.3 Gestures

As the user repeats these actions, the expectation is that users will naturally complete the gesture faster and faster, until only a quick flick is required. By having this new intermediate mode, users gradually get used to flicking towards the four directional commands while still showing the menu for guidance. Eloquent also enables expert users to select a word with a drag-press and flick to invoke the associated command, without the menu appearing. This behavior is largely inspired by marking menus, and aims for a gradual transition from novice to expert use.

6 CONCLUSION AND FUTURE WORK

It's understandable that existing mobile systems tried to reuse as much of the user experience from desktop as possible. However, it replaced the most basic action, a very accurate mouse click, with a more error prone finger tap. This forced mobile to invent new techniques to work around this problem, such as drag handles and offset magnifiers. These new additions make tapping more ambiguous, selection more problematic, and result in mobile text editing being more frustrating to users.

Eloquent is an interlocking system of supportive techniques: a draggable cursor, an inline magnifier, a drag-press gesture, a flat scrollable menu, and marking-menu style flick gestures. This keeps the same cursor-based text metaphor from the desktop but working together, these new techniques are more uniform, encourage more accuracy, and reward expert use. It's now possible for the user to target, select, and act on text with a more accurate and

continuous motion. While our work was informed by qualitative studies, and input from users helped us in the design process, these techniques require more formal evaluation. Future work will focus on controlled user studies of all techniques presented in this demonstration.

ACKNOWLEDGMENTS

The authors would like to thank Shumin Zhai for sponsoring this work, Philip Quinn for initial work on using barometric sensor data to sense finger pressure on touch screens, as well as Matt Jones, Simon Robinson, and Alex Olwal for their feedback on the paper and video demo.

REFERENCES

- [1] Shumin Zhai, Per Ola Kristensson. 2012. The word-gesture keyboard: reimagining keyboard interaction. *Communications of the ACM*, Volume 55, Issue 9 September 2012, pp 91–101.
- [2] Scheibel, J.-B., Pierson, C., Martin, B., Godard, N., Fuccella, V. and Isokoski, P. 2013. Virtual stick in caret positioning on touch screens. In *Proceedings of the 25th Conference on l'Interaction Homme-Machine*, p. 107.
- [3] Luis A. Leiva, Alireza Sahami, Alejandro Catala, Niels Henze, Albrecht Schmidt. 2015. Text Entry on Tiny QWERTY Soft Keyboards. *CHI '15: Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems* April 2015 Pages 669–678.
- [4] Anne Roudaut, Eric Lecolinet, Yves Guiard. 2009. MicroRolls: expanding touch-screen input vocabulary by distinguishing rolls vs. slides of the thumb. *CHI '09: Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* April 2009 Pages 927–936.
- [5] Mitchell Clark 2021 Apple brought back the beloved magnifying glass for selecting text in iOS 15. <https://www.theverge.com/2021/6/8/22524824/apple-ios-15-iphone-text-selection-magnifying-glass-reversal>
- [6] Allen, R.B.: Cognitive factors in the use of menus and trees: an experiment. *IEEE Journal on Selected Areas in Communications* SAC-1(2), 333–336 (1983)
- [7] Huy Viet Le, Sven Mayer, Maximilian Weiß, Jonas Vogelsang, Henrike Weingärtner, and Niels Henze. 2020. Shortcut Gestures for Mobile Text Editing on Fully Touch Sensitive Smartphones. *ACM Trans. Comput.-Hum. Interact.* 27, 5, Article 33 (August 2020), 38 pages.
- [8] Jingjie Zheng, Xiaojun Bi, Kun Li, Yang Li, and Shumin Zhai. 2018. M3 Gesture Menu: Design and Experimental Analyses of Marking Menus for Touchscreen Mobile Interaction. *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Paper 249, 1–14.